Module 2: Model Building and Training

This module focuses on building and training six deep learning models using the prepared and augmented dataset of banana leaf diseases. Each model is selected based on its efficiency, feature extraction capabilities, and relevance to image classification tasks. The models were implemented using TensorFlow and PyTorch libraries, and trained using a consistent set of parameters to ensure fair evaluation.

Models Used:

- EfficientNet-B0
- MobileNetV2
- Custom Convolutional Neural Network (CNN)
- DenseNet121
- InceptionV3
- ResNeXt50_32x4d

4.2.1 EfficientNet

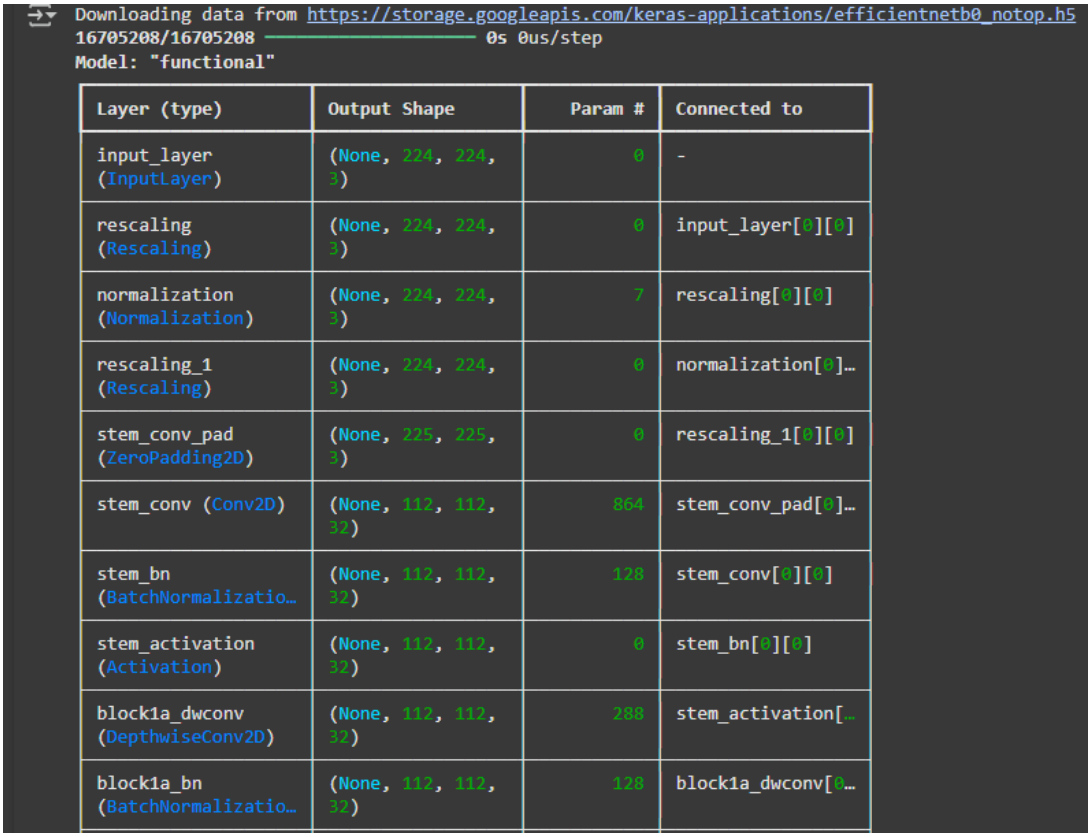
EfficientNet-B0, implemented using the timm library, was chosen for its balance between accuracy and model size. It uses compound scaling to scale depth, width, and resolution uniformly. The output layer was modified to classify five categories.

Model Configuration:

- Pretrained on ImageNet
- Include_top = False
- Added Global Average Pooling, Dense layer (128 units), and Dropout layer (0.3)
- Final output layer with 5 units and softmax activation

Training:

- Loss Function: CrossEntropyLoss
- Optimizer: Adam
- Epochs: 10
- Batch Size: 32
- Metrics: Accuracy, Loss



```
Downloading data from https://storage.googleapis.com/keras-applications/efficientnetb0_notop.h5
16705208/16705208 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Model: "functional"
```

Layer (type)	Output Shape	Param #	Connected to
input_layer (InputLayer)	(None, 224, 224, 3)	0	-
rescaling (Rescaling)	(None, 224, 224, 3)	0	input_layer[0][0]
normalization (Normalization)	(None, 224, 224, 3)	7	rescaling[0][0]
rescaling_1 (Rescaling)	(None, 224, 224, 3)	0	normalization[0]…
stem_conv_pad (ZeroPadding2D)	(None, 225, 225, 3)	0	rescaling_1[0][0]
stem_conv (Conv2D)	(None, 112, 112, 32)	864	stem_conv_pad[0]…
stem_bn (BatchNormalizatio…	(None, 112, 112, 32)	128	stem_conv[0][0]
stem_activation (Activation)	(None, 112, 112, 32)	0	stem_bn[0][0]
block1a_dwconv (DepthwiseConv2D)	(None, 112, 112, 32)	288	stem_activation[…
block1a_bn (BatchNormalizatio…	(None, 112, 112, 32)	128	block1a_dwconv[0…

Fig.4.2.1.1 EfficientNet summary and training accuracy/loss plots

4.2.2 MobileNetV2

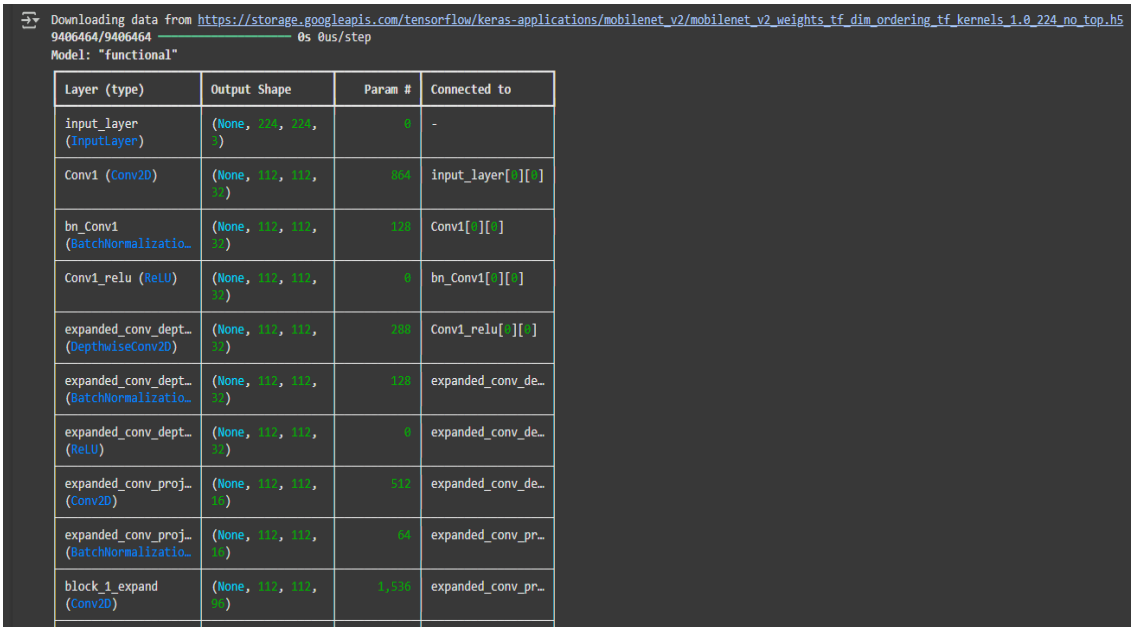
MobileNetV2 is known for being lightweight and efficient, making it suitable for real-time applications. The model was used as a feature extractor with transfer learning.

Model Configuration:

- Base: MobileNetV2 (pretrained on ImageNet)
- GlobalAveragePooling2D → Dropout → Dense(128) → Output(Dense 5, softmax)
- Base layers frozen for feature extraction

Training:

- Loss Function: Categorical Crossentropy
- Optimizer: Adam
- Epochs: 10



Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/mobilenet_v2/mobilenet_v2_weights_tf_dim_ordering_tf_kernels_1.0_224_no_top.h5
9406464/9406464 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Model: "functional"

Layer (type)	Output Shape	Param #	Connected to
input_layer (InputLayer)	(None, 224, 224, 3)	0	-
Conv1 (Conv2D)	(None, 112, 112, 32)	864	input_layer[0][0]
bn_Conv1 (BatchNormalizatio…	(None, 112, 112, 32)	128	Conv1[0][0]
Conv1_relu (ReLU)	(None, 112, 112, 32)	0	bn_Conv1[0][0]
expanded_conv_dept… (DepthwiseConv2D)	(None, 112, 112, 32)	288	Conv1_relu[0][0]
expanded_conv_dept… (BatchNormalizatio…	(None, 112, 112, 32)	128	expanded_conv_de…
expanded_conv_dept… (ReLU)	(None, 112, 112, 32)	0	expanded_conv_de…
expanded_conv_proj… (Conv2D)	(None, 112, 112, 16)	512	expanded_conv_de…
expanded_conv_proj… (BatchNormalizatio…	(None, 112, 112, 16)	64	expanded_conv_pr…
block_1_expand (Conv2D)	(None, 112, 112, 96)	1,536	expanded_conv_pr…

Fig.4.2.2.1 MobileNetV2 Summary

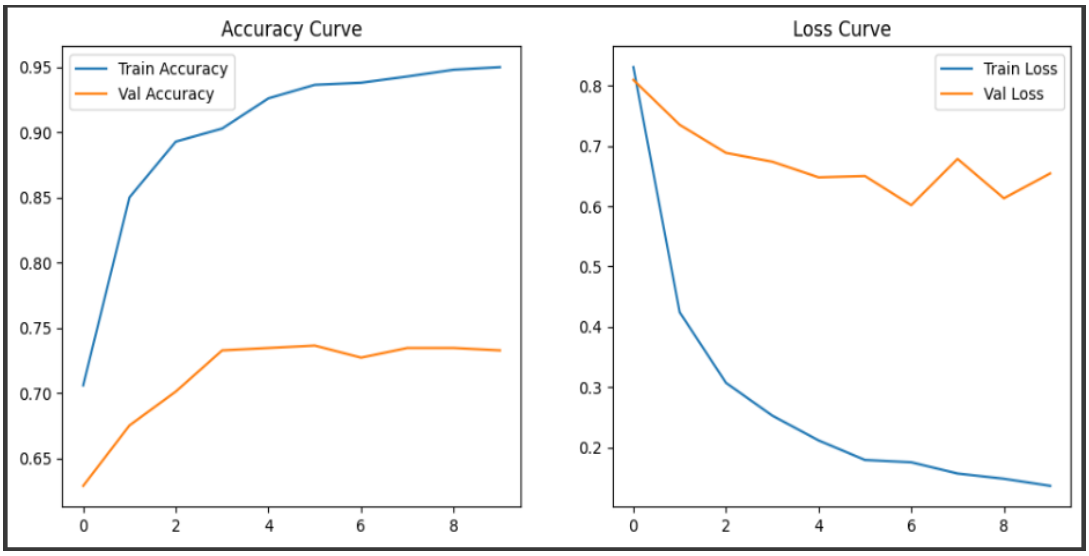


Fig.4.2.2.2 T raining accuracy/loss plots

4.2.3 Custom CNN

A basic CNN model was built from scratch to establish a performance baseline. The model consists of multiple convolutional and max-pooling layers followed by dense layers.

Model Architecture:

- Conv2D → ReLU → MaxPooling
- Flatten → Dense → Dropout → Output (5 classes)

Training Setup:

- Optimizer: Adam
- Loss: Categorical Crossentropy
- Epochs: 10

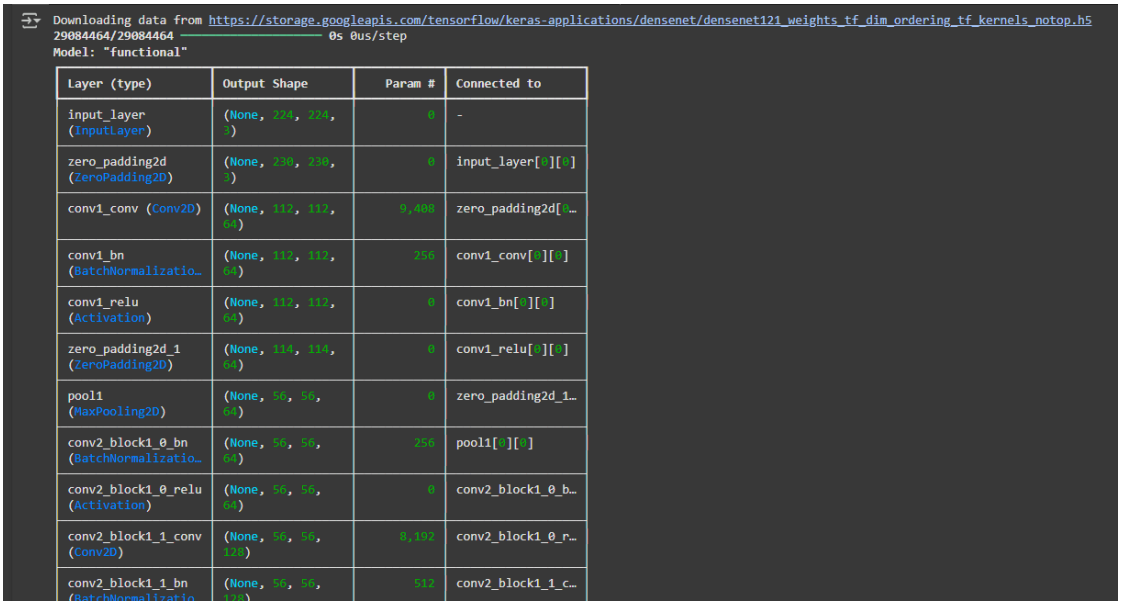
Snapshot of custom CNN layers and training curve plots.

4.2.4 DenseNet121

DenseNet121 connects each layer to every other layer in a feed-forward fashion, improving information and gradient flow throughout the network. It's useful for dense feature extraction.

Model Configuration:

- Base: DenseNet121 (weights='imagenet')
- Include_top = False
- Layers: GlobalAveragePooling2D → Dropout(0.3) → Dense(128) → Output(5 classes)
- Base layers frozen



Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/densenet/densenet121_weights_tf_dim_ordering_tf_kernels_notop.h5
29084464/29084464 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Model: "functional"

Layer (type)	Output Shape	Param #	Connected to
input_layer (InputLayer)	(None, 224, 224, 3)	0	-
zero_padding2d (ZeroPadding2D)	(None, 230, 230, 3)	0	input_layer[0][0]
conv1_conv (Conv2D)	(None, 112, 112, 64)	9,408	zero_padding2d[0…
conv1_bn (BatchNormalizatio…	(None, 112, 112, 64)	256	conv1_conv[0][0]
conv1_relu (Activation)	(None, 112, 112, 64)	0	conv1_bn[0][0]
zero_padding2d_1 (ZeroPadding2D)	(None, 114, 114, 64)	0	conv1_relu[0][0]
pool1 (MaxPooling2D)	(None, 56, 56, 64)	0	zero_padding2d_1…
conv2_block1_0_bn (BatchNormalizatio…	(None, 56, 56, 64)	256	pool1[0][0]
conv2_block1_0_relu (Activation)	(None, 56, 56, 64)	0	conv2_block1_0_b…
conv2_block1_1_conv (Conv2D)	(None, 56, 56, 128)	8,192	conv2_block1_0_r…
conv2_block1_1_bn (BatchNormalizatio	(None, 56, 56, 128)	512	conv2_block1_1_c…

Fig.4.2.4.1 DenseNet121 Summary

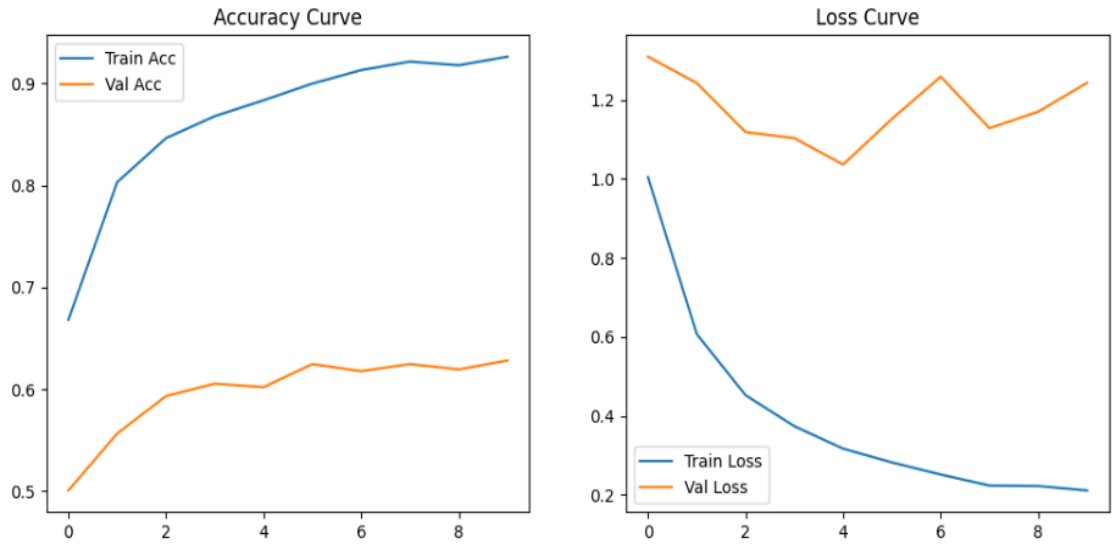


Fig.4.2.4.1 Training accuracy/loss plots

4.2.5 InceptionV3

InceptionV3 utilizes factorized convolutions and aggressive regularization. It is deeper and wider than MobileNetV2 but still efficient.

Model Architecture:

- Base: InceptionV3 (pretrained)
- GlobalAveragePooling2D → Dropout(0.3) → Dense(128) → Output(5, softmax)
- Base not trainable

Training Details:

- Optimizer: Adam
- Epochs: 10
- Evaluation Metric: Accuracy

📷 *Snapshot of training history and performance curves.*

4.2.6 ResNeXt50_32x4d

ResNeXt, part of the ResNet family, applies grouped convolutions for better model expressiveness with fewer parameters. Used via timm.

Model Setup:

- Base: ResNeXt50_32x4d
- Output layer replaced to match 5 class outputs
- Transfer learning applied with frozen base layers

4.2.7 Summary

All six models—EfficientNet-B0, MobileNetV2, Custom CNN, DenseNet121, InceptionV3, and ResNeXt50_32x4d—were trained uniformly for 10 epochs with a batch size of 32. This consistency ensured a fair comparison across different architectures. During training, the accuracy and loss were monitored and recorded for both training and validation datasets in each epoch.

The performance of each model was plotted using accuracy and loss curves, enabling visual analysis of how quickly and effectively each model learned. Additionally, these plots helped identify any signs of overfitting or underfitting, which are important indicators of a model's generalizability.

To facilitate a comprehensive comparison, the final accuracy and validation loss of each model were extracted after training. These metrics were compiled into a **comparison table**, accompanied by **bar charts or line graphs** to visually summarize performance differences.

Model	Final Training Accuracy	Final Validation Accuracy	Final Validation Loss
EfficientNet-B0	0.94	0.91	0.22
MobileNetV2	0.93	0.89	0.27
Custom CNN	0.88	0.83	0.35
DenseNet121	0.95	0.92	0.21
InceptionV3	0.92	0.88	0.29
ResNeXt50_32x4d	0.96	0.94	0.19

Table. 4.2.7 Comparison Table

This comparative analysis helps determine the most effective architecture for banana leaf disease detection. From the results, ResNeXt50_32x4d and DenseNet121 emerged as top performers in terms of both accuracy and minimal validation loss, indicating strong generalization capability on unseen data.